

Now I have comprehensive context on both the UX research and the existing codebase. Let me produce the PRD.

PRD: Force Trigger Automations

1. Overview

Automations on 5Mins.ai let admins define enrolment logic — audience rules, course sequences, delays, and recurrence — to automatically onboard employees into learning paths. Today, automations only evaluate at user registration. If a user slips through the cracks during onboarding (wrong role assigned at the time, delayed provisioning, data sync issue), the admin's only recourse is to manually enrol that user into each course one by one.

Force Trigger gives admins a way to "re-hire" an existing automation for a specific set of users on demand, without rebuilding the enrolment logic by hand. The automation runs with its full configuration intact — delay scheduling, recurrence, and course sequencing are all preserved. The only thing bypassed is the audience rules, since the admin's manual user selection replaces the rule filter.

2. Jobs To Be Done

Main Job

When an employee misses their intended automation window, the admin needs to retroactively apply that automation's full enrolment sequence to them — without manually recreating the logic course by course.

The admin is "hiring" Force Trigger to replace the current workaround of navigating to individual courses and manually enrolling users one by one. The manual workaround forces the admin to drop the automation's delay scheduling, recurrence, and sequencing — Force Trigger preserves all of it, because the automation runs with its own configuration.

Job Map

Stage	What the Admin Does Today	Where It Breaks Down
Define	Admin has already defined the automation (courses, rules, delays, recurrence).	No issue — automations are well-defined.
Locate	Admin identifies which users missed the automation by checking enrolment reports or getting complaints.	Works, but requires cross-referencing the People page and course enrolments manually.
Prepare	Admin opens each course in the automation and manually enrolls each affected user.	Tedious and error-prone. Delays, ordering, and recurrence are lost. The admin is "reverse-engineering" the automation by hand.
Confirm	Admin reviews their manual enrolments.	No holistic view — each course is enrolled separately. No confidence that sequencing is right.

Execute	Admin saves each enrolment individually.	Multiple clicks, multiple pages, multiple chances for mistakes.
Monitor	Admin checks back later to verify enrolments landed.	No centralized feedback — must check each course.
Resolve	If something went wrong, admin starts over from Locate.	No audit trail of the manual enrolment attempt.

Force Trigger collapses Prepare → Confirm → Execute into a single modal interaction: select users, review the automation's courses, trigger.

Related Jobs

- **Recover from onboarding errors** — When user data (role, region) was wrong at registration and has since been corrected, the admin needs to apply the automation that *should* have matched.
- **Re-enrol users for compliance refreshers** — When recurrence alone isn't sufficient and the admin needs to force a restart of a compliance training sequence mid-cycle.
- **Onboard users who joined before an automation existed** — When a new automation is created for existing employees who weren't captured by a "new employee" trigger.

Emotional & Social Dimensions

- **Confidence:** The admin wants to feel certain that triggering an automation will do exactly what the automation definition says — the right courses, in the right order, with the right delays. No guesswork.
- **Control:** The admin wants to feel empowered to fix onboarding gaps without filing a support ticket or asking engineering for help.
- **Trust:** The admin wants to trust that re-triggering won't destroy a user's existing progress or create data inconsistencies.
- **Competence:** The admin wants to appear effective to stakeholders — "I caught the gap and fixed it in 30 seconds" rather than "I'm still manually enrolling people."

3. Goals

1. **Zero-friction recovery:** An admin who discovers a missed automation should be able to force-trigger it for affected users in under 60 seconds, from the automations page, without navigating away.
2. **Full fidelity:** Force-triggered enrolments must replicate the automation's course sequence, delay scheduling, and recurrence — not just the course list.
3. **Transparency before action:** The admin must see exactly what courses will be enrolled, and be warned about re-triggers, *before* executing.
4. **Graceful failure handling:** The admin must know which specific users failed (not just "something went wrong") so they can investigate and retry.

4. Job Stories

Discovery & Access

- **When** I'm reviewing my automations list and realize a user was missed, **I want to** trigger that automation directly from the list row, **so I can** fix the gap without navigating to a different page or rebuilding the enrolment manually.

Understanding Impact

- **When** I open the Force Trigger modal, **I want to** see the full list of courses the automation will enrol users into (including delay information), **so I can** confirm this is the right automation before selecting any users.

User Selection

- **When** I'm selecting users to force-trigger, **I want to** search by name or email and see results instantly, **so I can** quickly find the specific people who need this automation.
- **When** I select a user who was previously triggered by this automation, **I want to** see a visual indicator (🕒) before I finalize, **so I can** make an informed decision about whether to re-trigger them.
- **When** I've selected multiple users, **I want to** see them as removable chips below the search input, **so I can** review and adjust my selection before executing.

Re-trigger Awareness

- **When** any of my selected users have been previously triggered by this automation, **I want to** see a non-blocking warning explaining that their enrolments will be reset, **so I can** proceed with full awareness of the consequences.

Execution & Feedback

- **When** I click "Trigger for X users," **I want** the automation to execute immediately without an additional confirmation step, **so I can** move fast — the modal already showed me what will happen and to whom.
- **When** all triggers succeed, **I want to** see a toast confirmation and have the modal close automatically, **so I can** move on to my next task.
- **When** some or all triggers fail, **I want to** see an in-modal alert listing the specific failed users, **so I can** investigate and retry without losing context.

5. Requirements

Functional Requirements

FR-1: Entry Point — : Menu on Automation Rows

ID	Requirement
FR-1.1	Each automation row's : overflow menu includes a "Force Trigger" option, positioned after existing actions (Edit, Delete).
FR-1.2	On inactive automations, the "Force Trigger" option is rendered in a disabled state (visible but non-interactive, with a tooltip explaining that the automation must be active to trigger).
FR-1.3	The "Force Trigger" option does not appear on the automation type template cards ("New Employee Automation" / "Existing Employee Automation") at the top of the Manage tab.
FR-1.4	Clicking "Force Trigger" opens the Force Trigger modal and passes the automation's id and name to it.

FR-2: Data Model Extension

ID	Requirement
FR-2.1	The AutomationRow interface is extended with a courses array field: courses: { id: string; name: string; delay: string; dueDate?: string }[].

FR-2.2	Mock data for all 14 existing automations is updated to include realistic courses[] entries. At least 2 automations should have empty courses[] to exercise the empty-state path.
--------	---

FR-3: Modal — Step 1: Course Summary (Shown First)

ID	Requirement
FR-3.1	The modal header displays "Force Trigger: {Automation Name}".
FR-3.2	A course summary section lists all courses in the automation with their delay values (e.g., "Course A — 0 days after trigger", "Course B — 7 days after previous course").
FR-3.3	The automation's rules are not shown in the modal.

FR-4: Modal — Step 2: User Search & Selection

ID	Requirement
FR-4.1	A search input below the course summary accepts text input for filtering by name or email.
FR-4.2	Search is client-side against the existing mockUsers array, matching against name and email fields (case-insensitive, substring match).
FR-4.3	Search results appear in a dropdown below the input, showing user name and email for each result.
FR-4.4	Users who have been previously triggered by this specific automation (matched via mockTriggers) display a 🕒 icon next to their name in the search results.
FR-4.5	Clicking a search result adds the user to the selection and the user appears as a removable chip below the search input.
FR-4.6	Already-selected users do not appear in the search dropdown.
FR-4.7	Each chip shows the user's name and an × button to deselect.
FR-4.8	There is no hard limit on the number of users that can be selected.

FR-5: Re-trigger Warning

ID	Requirement
FR-5.1	If any selected user has been previously triggered by this automation, an informational warning banner appears in the modal (e.g., "Some selected users have been triggered by this automation before. Their existing enrolments will be reset and new ones created. Previous progress is preserved in history.").

FR-5.2	The warning is informational only — it does not block the trigger action.
FR-5.3	The warning dynamically appears/disappears as users are added/removed from the selection.

FR-6: Execution

ID	Requirement
FR-6.1	The trigger button reads "Trigger for {N} users" and is disabled when no users are selected.
FR-6.2	Clicking the trigger button executes immediately — no additional confirmation dialog.
FR-6.3	During execution, the button enters a loading state (disabled + spinner or loading text).
FR-6.4	Success (all users): The modal closes and a toast notification displays "Automation triggered for {N} users."
FR-6.5	Full failure: An in-modal alert (not a toast) displays listing which users failed with a message like "Failed to trigger automation for the following users:" followed by user names. The modal remains open.
FR-6.6	Partial failure: Same in-modal alert, listing only the failed users. The modal remains open so the admin can review. Successful users are removed from the chip selection.
FR-6.7	For the prototype, execution always simulates success. A code marker (<code>// TODO: Replace with actual API call – handle partial failure</code>) is left at the execution callsite.
FR-6.8	The in-modal alert slot is rendered (conditionally visible) and wired up to state, so future integration only needs to set the error state.

FR-7: Activity Tab Integration

ID	Requirement
FR-7.1	On simulated success, new <code>TriggerRow</code> entries are added to the <code>mockTriggers</code> array with <code>triggeredAt</code> set to the current ISO timestamp.
FR-7.2	These new triggers appear in the Activity tab's table immediately (since data is in-memory).

6. Research & Best Practices

Industry Patterns for Manual Automation Triggers

Research into enterprise LMS platforms (LearnUpon, 360Learning, CYPHER Learning, Tutor LMS) reveals a consistent pattern: **automations that only fire at registration create a brittle onboarding funnel**. The industry responses include:

- **Group-based auto-enrollment** with dynamic rules that re-evaluate on attribute changes (LearnUpon).
- **Bulk CSV enrollment** as a manual bypass (Tutor LMS) — functional but disconnected from automation logic.
- **Prerequisite chain enrollment** where completion of one course triggers the next (TrainingSites) — doesn't help when the chain was never started.

Force Trigger is differentiated because it **reuses the existing automation definition** rather than requiring the admin to reconstruct the enrollment sequence manually. This is a less common but more powerful pattern — closer to Salesforce's "Run Flow" capability where an admin can manually invoke an existing automation for specific records.

Modal UX Best Practices

Research from NN/g, LogRocket, PatternFly, and Eleken converges on these principles relevant to the Force Trigger modal:

1. **Show consequences before confirmation** — The course summary must appear before user selection. The admin should understand *what* will happen before deciding *who* it happens to. This matches PatternFly's "change preview" pattern for bulk edit modals.
2. **Action-verb buttons** — "Trigger for 3 users" is significantly better than "Confirm" or "OK". Research shows users are more likely to read button text than modal body copy, so the button must encode the action and scope.
3. **Non-blocking warnings over blocking confirmations** — NN/g warns against overusing confirmation dialogs ("the cry-wolf problem"). Since re-triggering is a valid, intended action (not a destructive error), an informational banner is the right pattern — not a second confirmation modal.
4. **In-modal error detail over generic toasts** — For failure states, Eleken's bulk action guidelines recommend showing "successful vs. failed items with counts" inline. A toast that says "something failed" violates the admin's need to know *what* to fix.

Combobox + Chips Selection Pattern

Mobbin's analysis of 600+ real-world combobox implementations identifies the **combobox + chips** combination as the ideal pattern for multi-select scenarios. Key characteristics:

- **Chips provide instant visual confirmation** of what's selected.
- **x buttons on chips** allow easy deselection without re-opening the dropdown.
- **Dynamic filtering** as the user types narrows results without separate search/select steps.

UX References

Notion — "Add a Member" flow (documented on [Mobbin](#)): Notion's workspace member invitation uses a search input at the top of a modal. As the admin types, matching users appear in a dropdown. Selected members appear as a list below. This closely matches the Force Trigger user selection pattern — the key difference is Force Trigger uses chips (more compact for potentially larger selections) rather than full list rows.

Slack — Channel invite modal: Slack's "Add people to #channel" modal uses a combobox where selected users appear as inline chips within the input field itself. This is a more compact variant but can become unwieldy with many selections. The 5Mins pattern of showing chips *below* the input (as a separate section) is preferable for the potentially larger selection sets in Force Trigger.

Linear — Command modal ([Mobbin](#)): Linear's command palette demonstrates clean action-first hierarchy — the user sees available actions immediately, with search narrowing the options. This informs the modal's structure: courses (the "what") are shown first and prominently, before the user selection (the "who").

Asana — Share dashboard modal ([Mobbin](#)): Asana's share modal shows a clean input with email/name search, a dropdown of matching results, and a clear action button. The pattern of showing contextual information (what you're sharing) above the people selector maps directly to Force Trigger's course-summary-above-user-search layout.

PatternFly — Modal design guidelines ([PatternFly](#)): PatternFly recommends danger buttons for destructive actions and primary buttons for constructive ones. Force Trigger is constructive (creating enrolments), so the primary button style (`--primary`) from the existing design system is appropriate. The warning banner for re-triggers should use an informational style, not danger.

Existing Codebase Patterns to Reuse

The existing 5Mins codebase provides strong patterns to follow:

- **ConfirmModal** component for the modal shell (overlay, ESC handling, click-outside-to-close, 560px width, fade-in animation).
- **People page search pattern** for client-side name/email filtering against a user list.
- **Row action : menu** pattern already exists on automation rows (Edit, Delete) — Force Trigger is added as a third option.
- **Toast component** (`/src/components/Toast/`) for success notifications.
- **Deletion confirmation modal** in Automations.tsx for the "type to confirm" pattern — though Force Trigger intentionally doesn't use this pattern (the action is constructive, not destructive).

7. Plan of Action

Phase 1: Data Model & Entry Point

- ☐ Extend `AutomationRow` interface with `courses: { id: string; name: string; delay: string; dueDate?: string }[]`
- ☐ Update all 14 `mockAutomations` entries with realistic `courses[]` data (every automation has ≥ 1 course — empty arrays are not a valid state)
- ☐ Add "Force Trigger" option to the `: row action` menu — enabled on active automations, disabled (with tooltip) on inactive automations
- ☐ Add state variable to track which automation is being force-triggered (`forceTriggerAutomation: AutomationRow | null`)
- ☐ Wire menu click to open the Force Trigger modal (pass automation data)

Phase 2: Force Trigger Modal — Layout & Course Summary

- ☐ Create modal component using existing `ConfirmModal` shell
- ☐ Implement modal header: "Force Trigger: {Automation Name}"
- ☐ Build course summary list showing each course with its delay value
- ☐ Add CSS following `.force-trigger-*` BEM naming, using design tokens

Phase 3: User Search & Selection

- ☐ Add search input with iconsax `SearchNormal1` icon (matching People page pattern)
- ☐ Implement client-side filtering of `mockUsers` by name and email (case-insensitive substring)
- ☐ Build search results dropdown showing name + email per row
- ☐ Add  icon for users previously triggered by this automation (cross-reference `mockTriggers`)
- ☐ Implement chip-based selection: clicking a result adds a chip, chips have `x` to remove
- ☐ Exclude already-selected users from the dropdown results

- ☐ Implement click-outside-to-close for the search dropdown

Phase 4: Re-trigger Warning & Execution

- ☐ Add informational warning banner that appears when any selected user has prior triggers for this automation
- ☐ Implement dynamic "Trigger for {N} users" button with disabled state when selection is empty
- ☐ Add loading state to the trigger button during execution
- ☐ Implement success path: close modal + show toast ("Automation triggered for {N} users")
- ☐ Build in-modal error alert slot (conditionally rendered) for failure/partial failure display
- ☐ Wire error alert to list specific failed user names
- ☐ On success, remove successful users' chips; on partial failure, keep failed users' chips
- ☐ Add `// TODO: Replace with actual API call – handle partial failure` code marker
- ☐ Simulate success in prototype (all triggers succeed)

Phase 5: Activity Tab Integration & Polish

- ☐ On simulated success, append new `TriggerRow` entries to `mockTriggers` with current timestamp
- ☐ Verify new triggers appear in Activity tab table immediately
- ☐ Add keyboard accessibility: ESC to close, focus trap within modal, ARIA attributes
- ☐ Test all states: empty courses, no users selected, re-trigger warning on/off, success toast
- ☐ Ensure modal scroll behavior works when course list or chip list is long

8. Risks & Mitigations

Risk	Impact	Mitigation
Admin triggers wrong automation — clicks Force Trigger on the wrong row by accident.	Users enrolled in incorrect courses.	The modal's course summary (shown first, before user selection) acts as a safety net — the admin sees what will happen before selecting anyone. Automation name is in the modal header.
Re-trigger data loss perception — Admin fears re-triggering will destroy a user's progress and hesitates to use the feature.	Feature underutilization; admin falls back to manual enrolment.	Clear warning copy explicitly states "Previous progress is preserved in history." Use informational (not danger) styling to signal this is safe.
Modal overload with many courses — Automations with 10+ courses create a long scrollable summary that pushes user search below the fold.	Admin misses courses in the list or finds the modal awkward.	Make the course summary section scrollable with a max-height, keeping the search input and trigger button always visible.
Client-side search won't scale — Prototype uses client-side filtering; real orgs may have 50k+ users.	Performance degradation when backend is integrated.	Code marker (<code>// TODO: Replace with server-side search + debounce</code>) at the filter callsite. Design the search input to

		accommodate a loading spinner for future async results.
No undo for force triggers — Once triggered, there's no "undo" button or grace period.	Admin triggers the wrong users and must manually un-enroll them.	The two-phase modal (see courses first, then select users) plus the dynamic button label ("Trigger for 3 users") reduces accidental execution. Future iteration could add an undo toast with a short grace period.
Prototype mock data diverges from real backend behavior — Mock always-succeeds doesn't test failure paths.	Failure UI is untested in real conditions.	Wire up the error alert slot and state management now. Add a <code>?demo=force-trigger-fail</code> URL param to simulate partial failure for design review.
Chip area overflow — Admin selects 50+ users; chip area becomes unwieldy.	Modal becomes unusable; admin can't see courses or trigger button.	Set a max-height on the chip container with scroll, and show a count badge (e.g., "+42 more") when chips overflow. Keep trigger button pinned at the modal footer.